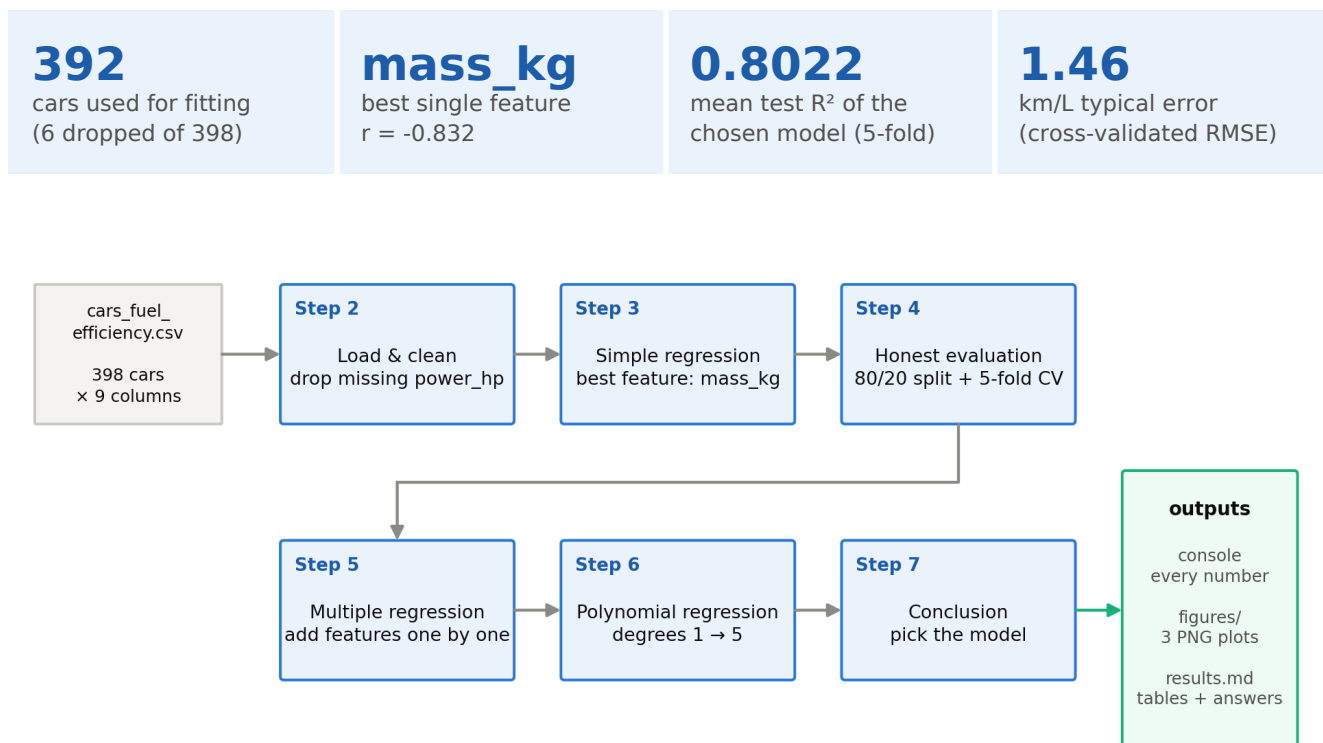


How lab3.py works

A visual, step-by-step walkthrough of the linear & polynomial regression lab: what every part of the code does, why it is written that way, and what the results mean.

Abror Imamsidikov · September 2026



Everything runs top-to-bottom from a single command: `python lab3.py`

Figure 1 — The whole script at a glance: one CSV in, six lab steps run in order, three kinds of output.

Heads-up. All numbers in this document were produced by running the current code with `RANDOM_STATE = 1234` (set in `lab/config.py`). That value is still the placeholder; with the real last-4 digits of the student ID, every number shifts slightly, but the code, the charts' shapes and the reasoning stay the same.

CONTENTS

What's inside

#	Section	What you will learn
1	The big picture	What the script does and how to run it
2	Code map	The files, what each one does, and how they connect
3	Step 2 · Load & clean	Reading the CSV and dropping missing values
4	Step 3 · Simple linear regression	Picking a feature and fitting a line
5	Step 4 · Honest evaluation	Train/test split, metrics, cross-validation
6	Step 5 · Multiple regression	Adding features one at a time
7	Step 6 · Polynomial regression	Curves, scaling and overfitting
8	Step 7 · Conclusion	Choosing the model to keep
9	Self-writing answers	How results.md text is generated from numbers
10	Cheat sheet & sources	Metric formulas, glossary, references

SECTION 1

The big picture

The code is split into small files so each one does one job. `lab3.py` is only the **entry point**: its `main()` function calls one module per lab step, in the order of the lab sheet. Each step prints its numbers, adds its part of the report, and returns a small result object that later steps use. At the very end the report is written to `results.md`.

Output	Where	What is in it
Console	terminal	every number the lab asks for, step by step
Plots	<code>figures/</code>	<code>step3_scatter.png</code> , <code>step3_fit.png</code> , <code>step6_r2_vs_degree.png</code>
Report	<code>results.md</code>	all tables plus the written answers for Steps 4–7

How to run it

```
lab3.py · lines 9–10 · from the module docstring
9 source .Imamsidikov_ML_lab_3/bin/activate
10 python lab3.py
```

The libraries are the four pinned in `requirements.txt`: **pandas** (tables), **numpy** (maths), **matplotlib** (plots) and **scikit-learn** (the models and evaluation tools).

SECTION 2

Code map

Every file is short (under ~110 lines) and has one job. The picture below shows who calls whom.

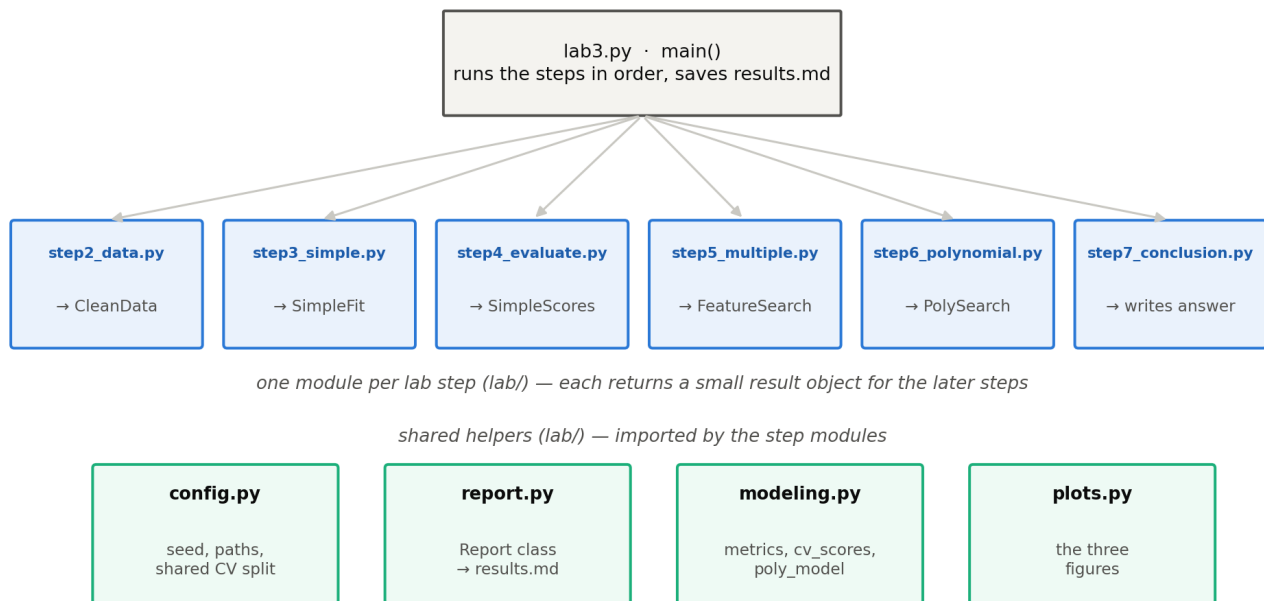


Figure 2 — How the files fit together. `lab3.py` calls the six step modules in order; each step uses the shared helpers at the bottom.

File	Lines	What is inside
<code>lab3.py</code>	57	entry point: <code>main()</code> runs Steps 2-7 and saves the report
<code>lab/config.py</code>	25	the seed, file paths, <code>MIN_GAIN</code> and the shared 5-fold split <code>CV</code>
<code>lab/report.py</code>	35	<code>Report</code> class that collects the markdown for <code>results.md</code> ; <code>code_list</code>
<code>lab/modeling.py</code>	41	<code>regression_metrics</code> , <code>cv_scores</code> , <code>poly_model</code>
<code>lab/plots.py</code>	42	the three figures saved to <code>figures/</code>
<code>lab/step2_data.py</code>	42	load the CSV, drop missing rows
<code>lab/step3_simple.py</code>	58	pick the best feature, fit the line
<code>lab/step4_evaluate.py</code>	84	train/test split, metrics, 5-fold CV
<code>lab/step5_multiple.py</code>	106	add features one by one
<code>lab/step6_polynomial.py</code>	104	degrees 1-5 with scaling
<code>lab/step7_conclusion.py</code>	76	choose the model, write the conclusion

The entry point

Reading `main()` tells you the whole story of the lab in six lines. Notice how each step's result is passed on to the steps that need it:

```
lab3.py · lines 44–49
44 data = step2_data.run(report)
45 fit = step3_simple.run(report, data.df)
46 simple = step4_evaluate.run(report, fit)
47 search = step5_multiple.run(report, data.df, fit)
48 poly = step6_polynomial.run(report, fit)
49 step7_conclusion.run(report, data, fit, simple, search, poly)
```

Result objects

Each step returns a small **frozen dataclass** — a named, read-only bundle of values — instead of leaving variables lying around. For example, Step 3 hands the chosen feature and the data to Steps 4–6:

```
lab/step3_simple.py · lines 13–18
13 @dataclass(frozen=True)
14 class SimpleFit:
15     best: str # the most correlated feature
16     numeric_features: list[str] # all numeric features, best-correlated first
17     X: pd.DataFrame # df[[best]]
18     y: pd.Series # the target
```

Settings that control everything

```
lab/config.py · lines 6–25
6 # Step 4: last 4 digits of my student ID.
7 # PLACEHOLDER – replace 1234 with the real digits and re-run the script.
8 RANDOM_STATE = 1234
9 PLACEHOLDER_STATE = 1234
10
11 ROOT = Path(__file__).resolve().parent.parent
12 DATA_PATH = ROOT / "data" / "cars_fuel_efficiency.csv"
13 FIG_DIR = ROOT / "figures"
14 RESULTS_PATH = ROOT / "results.md"
15
16 TARGET = "fuel_efficiency_km_per_l"
17 DEGREES = range(1, 6)
18
19 # In the written answers, a change in mean test  $R^2$  smaller than this counts as
20 # "no real improvement" (Steps 5–7).
21 MIN_GAIN = 0.005
22
23 # The same shuffled 5-fold split is reused in Steps 4, 5 and 6 so their  $R^2$  values
24 # are directly comparable. Shuffling matters: the CSV is ordered by model year.
25 CV = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
```

- **RANDOM_STATE** — the seed. It decides which cars land in the test set and how rows are shuffled into folds, so results are **reproducible** and personal to the student.
- **MIN_GAIN = 0.005** — the threshold used when the script writes its answers: a change in R^2 smaller than this is treated as “no real improvement”.
- **CV** — **one** shuffled 5-fold split created once and reused by Steps 4, 5 and 6. Because every model is scored on exactly the same folds, their R^2 values are directly comparable. Shuffling matters because the CSV is sorted by model year.

The shared helpers

Helper	File	What it does
<code>report.section(title)</code>	report.py	prints a banner and starts a “## ...” heading
<code>report.table(header, rows)</code>	report.py	adds a markdown table
<code>report.paragraph(*sentences)</code>	report.py	joins sentences into one paragraph
<code>code_list(names)</code>	report.py	formats names as <code>`a`</code> , <code>`b`</code> and <code>`c`</code>
<code>regression_metrics(y, pred)</code>	modeling.py	MAE, MSE, RMSE and R^2 for one set of predictions
<code>cv_scores(model, X, y)</code>	modeling.py	5-fold CV once: train R^2 , test R^2 , test RMSE per fold
<code>poly_model(degree)</code>	modeling.py	scale → polynomial → linear-regression pipeline
<code>plot_simple_fit(...)</code>	plots.py	scatter plot, then the same plot with the fitted line
<code>plot_r2_vs_degree(...)</code>	plots.py	train and test R^2 against degree

lab/modeling.py · lines 24–32 · the cross-validation workhorse

```

24 def cv_scores(model: BaseEstimator, X: pd.DataFrame, y: pd.Series) -> dict[str, np.ndarray]:
25     """Per-fold train  $R^2$ , test  $R^2$  and test RMSE on the shared 5-fold split."""
26     scores = cross_validate(model, X, y, cv=CV, return_train_score=True,
27                             scoring={"r2": "r2", "rmse": "neg_root_mean_squared_error"})
28     return {
29         "train_r2": scores["train_r2"],
30         "test_r2": scores["test_r2"],
31         "test_rmse": -scores["test_rmse"],
32     }

```

Note. `cross_validate` fits a **fresh copy** of the model on 4 folds and scores it on the 5th, five times over. Asking for two scorers (R^2 and RMSE) in one call means the folds are only fitted once. scikit-learn reports RMSE as a *negative* number (so that “higher is better” for every scorer), which is why the helper flips its sign.

SECTION 3 · STEP 2

Load & clean the data

The script reads the CSV into a pandas DataFrame and prints its shape, column types, summary statistics and the number of missing values per column. Only `power_hp` has gaps, so those rows are removed before any model sees the data (scikit-learn models cannot handle missing values).

lab/step2_data.py · lines 28–37

```

28 missing = df[df["power_hp"].isna()]
29 print("\nRows with missing power_hp:")
30 print(missing[["model", "year", "region"]])
31
32 rows_before = len(df)
33 df = df.dropna(subset=["power_hp"])
34 rows_dropped = rows_before - len(df)
35 share_dropped = f"{rows_dropped / rows_before:.2%}"
36 print(f"\nDropped {rows_dropped} of {rows_before} rows with missing power_hp ")
37     f"({share_dropped}); {len(df)} rows remain.")

```

398

rows in the CSV

6

rows with missing
power_hp

1.51%

share of the data
dropped

392

rows used by every
model

The dataset: 398 cars from model years 1970–1982, converted to metric units. Inputs are `year`, `cylinders`, `engine_litres`, `power_hp`, `mass_kg`, `accel_0_100_s` and `region`; the target is

`fuel_efficiency_km_per_l.missing` is saved *before* dropping so that Step 7 can say which years and regions the dropped cars came from.

SECTION 4 · STEP 3

Simple linear regression

Choosing the feature. `df.corr()` measures how strongly each numeric column moves with fuel efficiency (from -1 to $+1$). The script sorts by the *absolute* value, because a strong negative relationship is just as useful as a strong positive one, and picks the first.

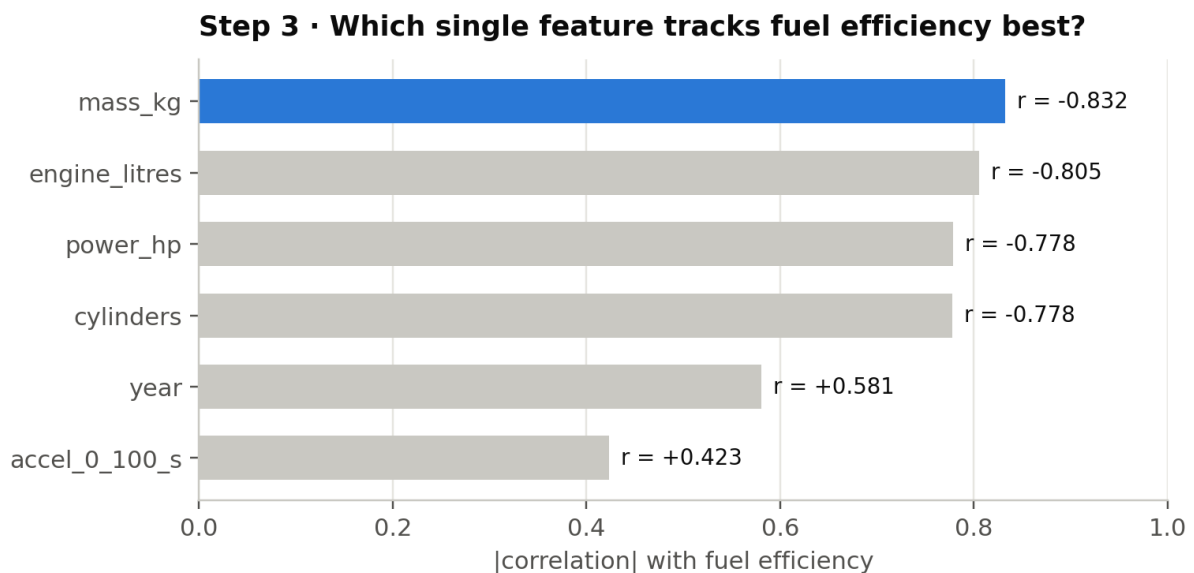


Figure 3 — Heavier cars use more fuel: `mass_kg` has the strongest correlation ($r = -0.832$). The sorted list is reused in Step 5 as the order in which features are added.

```
lab/step3_simple.py · lines 39–41
39 model = LinearRegression()
40 model.fit(X, y)           # X: shape (n, 1); y: shape (n,)
41 print(model.intercept_, model.coef_)
```

Fitting. `LinearRegression` finds the straight line that makes the sum of squared vertical distances between the cars and the line as small as possible (ordinary least squares). The intercept θ_0 is stored in `model.intercept_` and the slope θ_1 in `model.coef_[0]`.

$$\hat{y} = \theta_0 + \theta_1 \cdot x = \mathbf{19.6470} + \mathbf{(-0.007167)} \cdot \text{mass_kg}$$

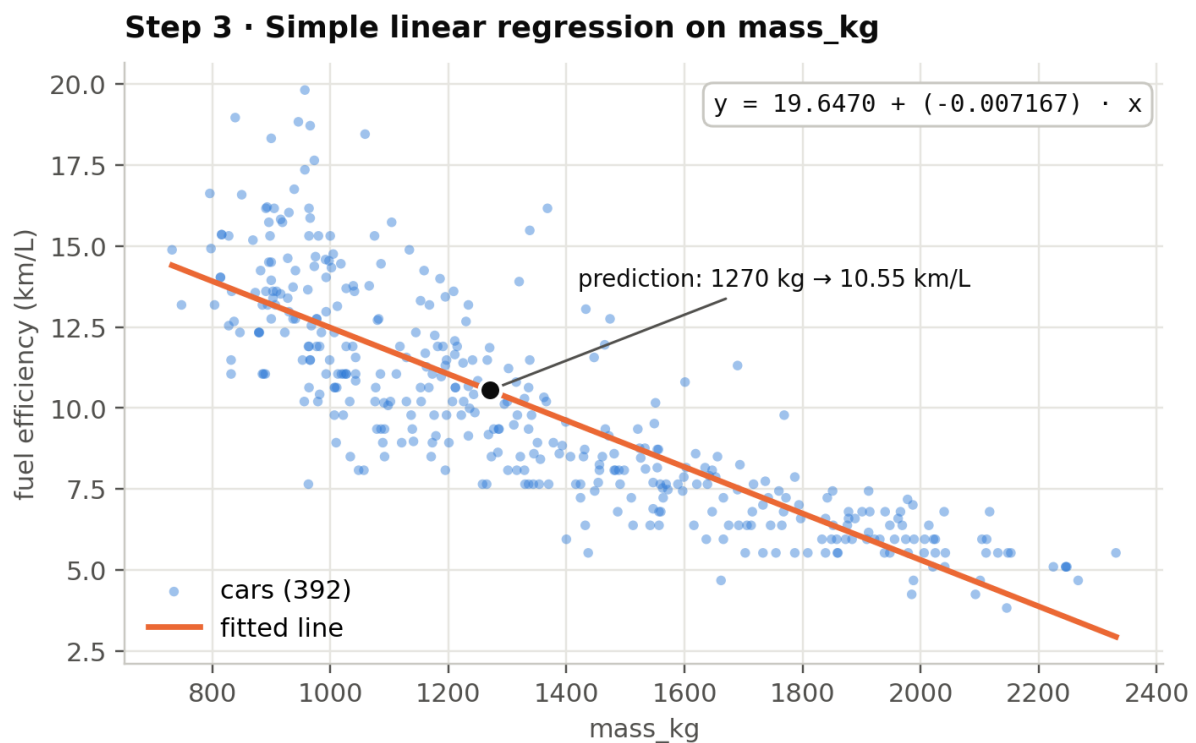


Figure 4 — The fitted line over all 392 cars, and the example prediction the script makes.

Reading the numbers. The slope -0.007167 means each extra kilogram costs about 0.0072 km/L, i.e. roughly **0.72 km/L per 100 kg**. For the example the script picks the median mass rounded to a round number (1270 kg):

$$19.6470 + (-0.007167) \times 1270 = \mathbf{10.55 \text{ km/L}}$$

Note. The line is fitted on *all* cars here because Step 3 is about understanding the relationship. Honest evaluation on unseen cars starts in Step 4.

SECTION 5 · STEP 4

Evaluate the model honestly

A model's score on the cars it was fitted to is always optimistic. Step 4 measures performance on cars the model **has not seen**, in two ways.

A. One train/test split (Step 4)

TRAIN 80% (313 cars) → .fit()

TEST 20% (79)

The model never sees the orange cars while fitting, so its score on them is an honest preview of new cars.

B. 5-fold cross-validation (rows shuffled first; Steps 4-6 reuse the same folds)

Round 1	test	train	train	train	train	test $R^2 = 0.6801$
Round 2	train	test	train	train	train	test $R^2 = 0.6546$
Round 3	train	train	test	train	train	test $R^2 = 0.6980$
Round 4	train	train	train	test	train	test $R^2 = 0.7373$
Round 5	train	train	train	train	test	test $R^2 = 0.6583$
						mean 0.6857 std 0.0302

Figure 5 — A: a single 80/20 split. B: 5-fold cross-validation — every car is used for testing exactly once, and the five test scores are averaged. Fold R^2 values are the real ones from this run.

lab/step4_evaluate.py · lines 25–30

```

25 X_train, X_test, y_train, y_test = train_test_split(
26     fit.X, fit.y, test_size=0.2, random_state=RANDOM_STATE
27 )
28 n_train, n_test = len(X_train), len(X_test)
29 shared_rows = len(X_train.index.intersection(X_test.index))
30 split_model = LinearRegression().fit(X_train, y_train)

```

The line `shared_rows = ...` is a safety check: it confirms that no car is in both sets (the answer is 0). The model is fitted with `.fit(X_train, y_train)` only, then `regression_metrics` scores it on each set.

Metric	Train (313 cars)	Test (79 cars)	Meaning
MAE	1.3633	1.4946	average size of the error, km/L
MSE	3.1929	4.1105	average squared error (punishes big misses)
RMSE	1.7869	2.0274	$\sqrt{\text{MSE}}$ — back in km/L
R^2	0.6948	0.6801	share of the variation explained (1 = perfect)

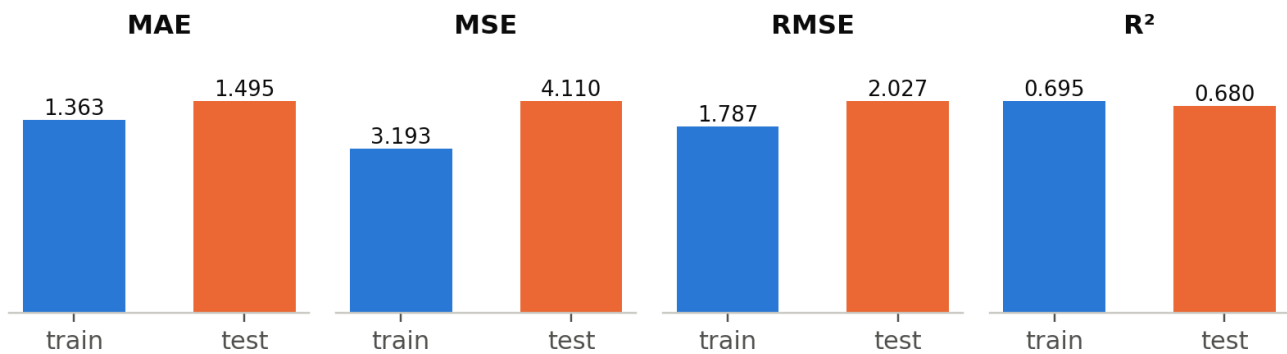
Step 4 · Error on the training cars vs the unseen test cars (lower is better, except R^2)

Figure 6 — Training errors are a little smaller than test errors: the expected pattern, and the lab's checkpoint passes.

Cross-validation. A single split depends on luck — which 79 cars happened to be picked for the test set. Five folds give five test scores; their mean (**0.6857**) is a steadier estimate, and the standard deviation (**0.0302**) shows how much the score moves from one split to another.

lab/step4_evaluate.py · lines 46–48

```
46 scores = cv_scores(LinearRegression(), fit.X, fit.y)
47 folds = scores["test_r2"]
48 result = SimpleScores(folds.mean(), folds.std(), scores["test_rmse"].mean())
```

Key idea. For some seeds the test score is *higher* than the training score. That is not a bug: a straight line has only two parameters and cannot memorise 313 cars, so the gap is mostly luck of the split. The script detects this case and writes a different explanation.

SECTION 6 · STEP 5

Multiple linear regression

Now the model uses several inputs at once: $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots$. The loop adds features **in order of their correlation** (best first) and re-runs the same 5-fold cross-validation after each addition.

lab/step5_multiple.py · lines 24–37

```
24 # One row per model: the k best-correlated features, for k = 1, 2, ...
25 rows = []
26 for k in range(1, len(fit.numeric_features) + 1):
27     features = fit.numeric_features[:k]
28     scores = cv_scores(LinearRegression(), df[features], fit.y)
29     rows.append({
30         "features": features,
31         "added": features[-1],
32         "mean_r2": scores["test_r2"].mean(),
33         "std_r2": scores["test_r2"].std(),
34         "rmse": scores["test_rmse"].mean(),
35     })
36 table = pd.DataFrame(rows)
37 table["change"] = table["mean_r2"].diff() # NaN for the first row
```

Each loop iteration stores one row; the rows become a DataFrame, and `.diff()` subtracts each row's R^2 from the previous one to get the “change” column.

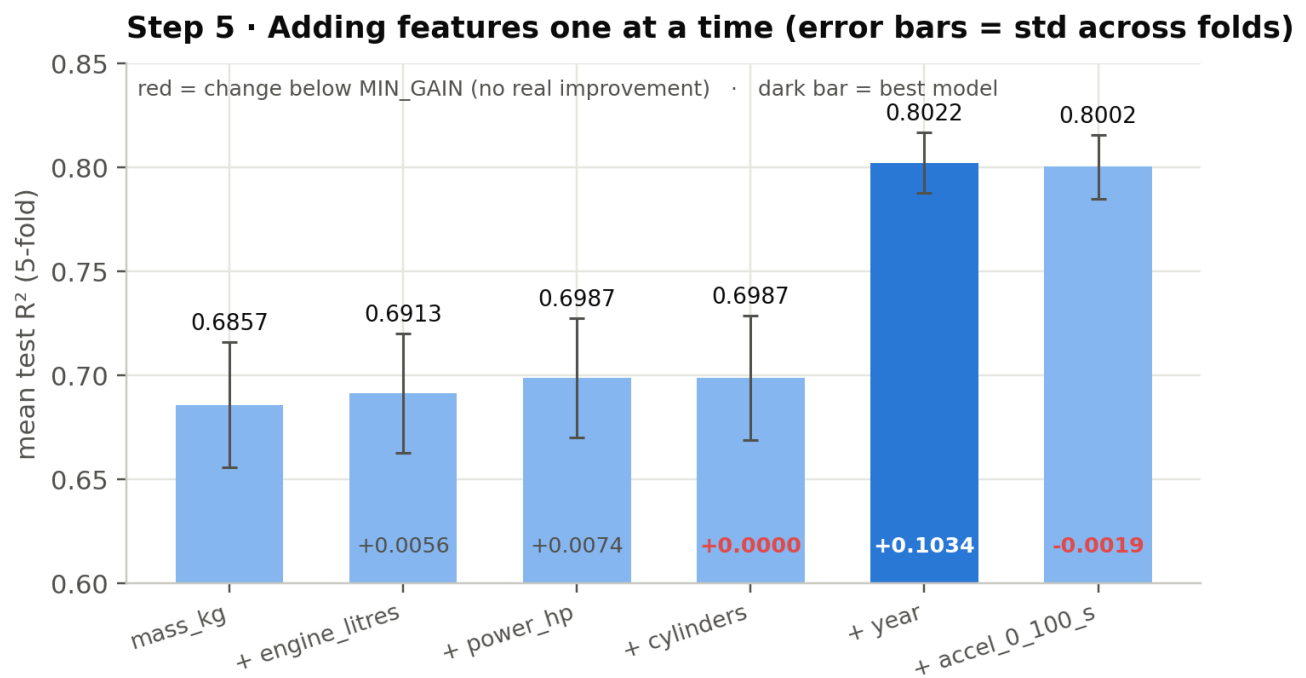


Figure 7 — Mean test R^2 after each addition. Two additions (red) change it by less than MIN_GAIN; adding year gives the big jump.

Features used (cumulative)	Mean test R^2	Std	Change
mass_kg	0.6857	0.0302	—
+ engine_litres	0.6913	0.0286	+0.0056
+ power_hp	0.6987	0.0288	+0.0074
+ cylinders	0.6987	0.0300	+0.0000
+ year	0.8022	0.0146	+0.1034
+ accel_0_100_s	0.8002	0.0155	-0.0019

Why didn't every feature help? Look at how the inputs relate to *each other*:

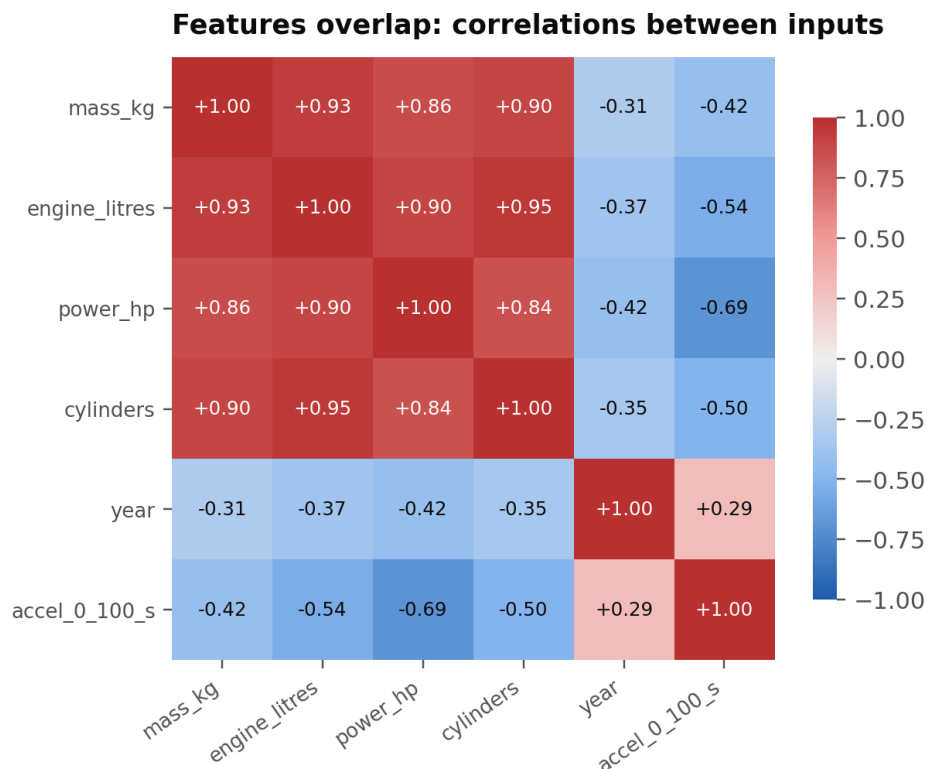


Figure 8 — mass, engine size, power and cylinders are all strongly correlated (0.84–0.95): they mostly describe the same thing, “a big car”. year is almost independent of them.

- **cylinders** (+0.0000) repeats what **engine_litres** already says ($r = +0.95$), so it adds nothing new.
- **year** ranks only 5th by correlation but gives **+0.1034**: it carries information none of the size columns have — newer cars were more efficient.
- **accel_0_100_s** slightly *lowers* the score (-0.0019): one more coefficient to estimate from the same cars, with no new information.

Key idea. A feature's value depends on what the model already contains, not on its own correlation with the target. That is why the script tests every addition on held-out data instead of throwing every column in.

SECTION 7 · STEP 6

Polynomial regression

Back to the single feature `mass_kg`, but now the model may bend: a degree- d polynomial is $\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_d x^d$. It is still “linear regression” — linear in the θ 's — the script just feeds it extra columns $x^2, x^3, \dots$. This is done with a **pipeline**:

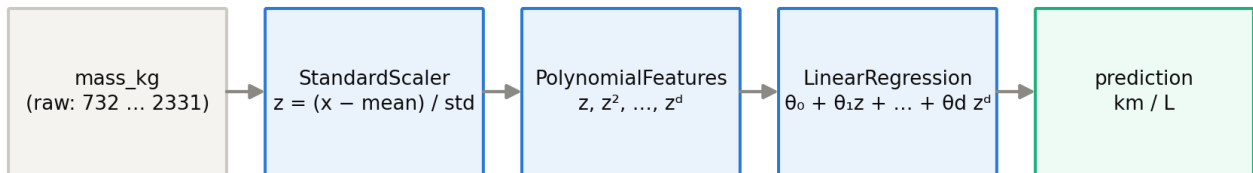


Figure 9 — What `poly_model(degree)` builds. Each stage's output is the next stage's input; the whole chain is refitted inside every CV fold.

lab/modeling.py · lines 35–41

```

35 def poly_model(degree: int) -> Pipeline:
36     """Scale first, so high powers of the raw feature don't break least squares."""
37     return make_pipeline(
38         StandardScaler(),
39         PolynomialFeatures(degree=degree, include_bias=False),
40         LinearRegression(),
41     )

```

Heads-up. Why scale first? Raw masses reach 2331 kg, and $2331^5 \approx 6.9 \times 10^{16}$. Numbers that far apart make least squares numerically unstable (you would see train R^2 drop as the degree rises, which is impossible in exact maths). `StandardScaler` turns mass into z-scores around 0 first, so z^5 stays small. Because the scaler sits *inside* the pipeline, it is re-fitted on the training folds only — the test fold never leaks into it.

lab/step6_polynomial.py · lines 23–29

```

23 # One row per degree, indexed by the degree itself.
24 rows = {}
25 for degree in DEGREES:
26     scores = cv_scores(poly_model(degree), fit.X, fit.y)
27     rows[degree] = {"train_r2": scores["train_r2"].mean(),
28                   "test_r2": scores["test_r2"].mean()}
29 poly = pd.DataFrame.from_dict(rows, orient="index")

```

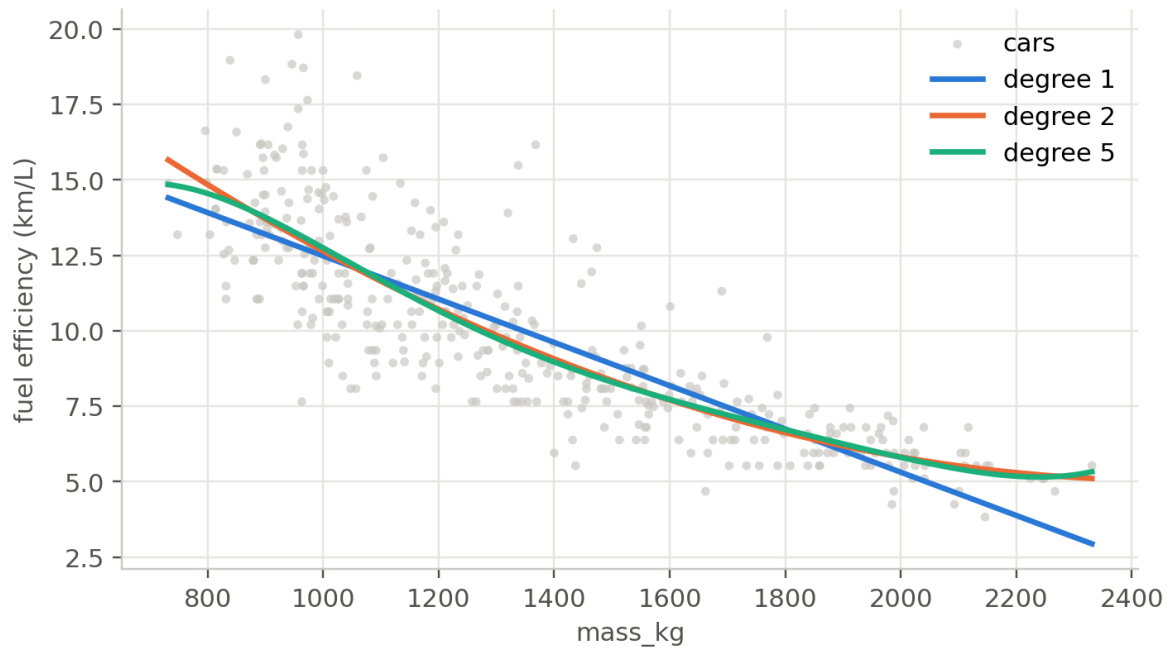
Step 6 · Straight line vs curves of degree 2 and 5

Figure 10 — Degree 1 is a straight line; degree 2 adds a gentle bend (efficiency levels off for heavy cars); degree 5 is almost identical to degree 2 — the extra freedom finds no new pattern.

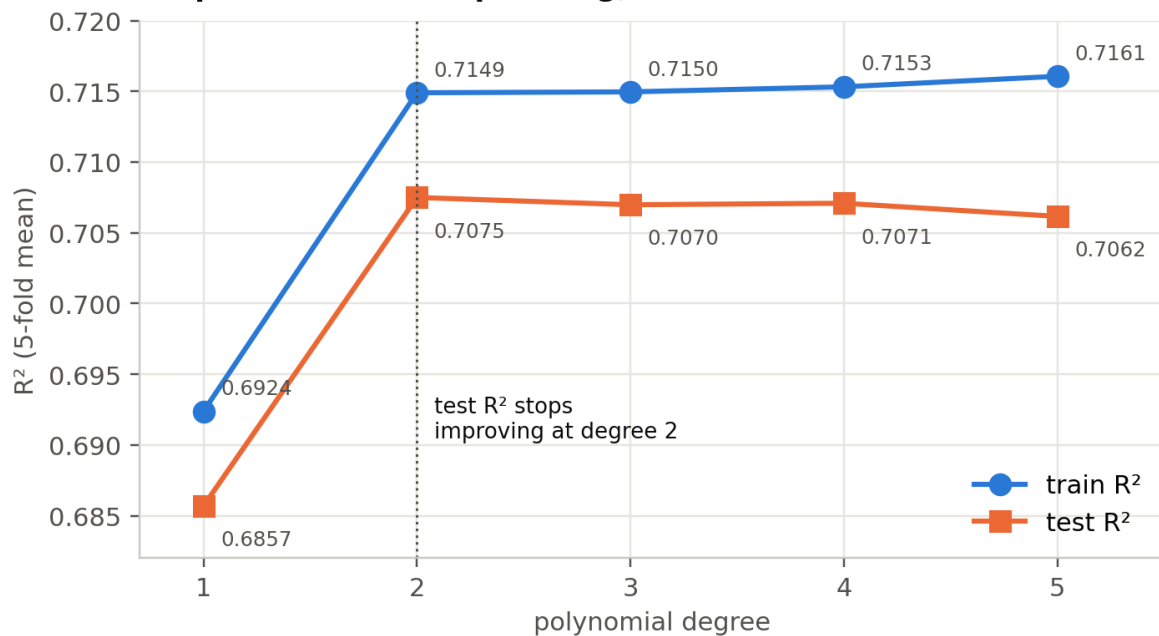
Step 6 · Train R^2 keeps rising, test R^2 flattens

Figure 11 — The same chart the script saves as figures/step6_r2_vs_degree.png, with the values labelled.

Degree	Train R^2	Test R^2	Test change
1	0.6924	0.6857	—
2	0.7149	0.7075	+0.0218
3	0.7150	0.7070	-0.0005
4	0.7153	0.7071	+0.0001
5	0.7161	0.7062	-0.0009

Where it stops improving: degree 2. Train R^2 keeps climbing because a higher-degree polynomial contains every lower-degree one — on the training cars it can only match or beat it. But the extra bends chase random scatter in the training cars, and that scatter does not repeat in the held-out fold, so test R^2 flattens and then slips. This is **overfitting** in miniature.

The script finds the stopping point with one line: the *lowest* degree whose test R^2 is within `MIN_GAIN` of the best.

```
lab/step6_polynomial.py · lines 43–43
```

```
43 stop_degree = poly.index[poly.test_r2 >= poly.test_r2.max() - MIN_GAIN][0]
```

SECTION 8 · STEP 7

Conclusion — which model to keep

The script compares the best model from each family, all scored on the same five folds, and chooses the row of Step 5 with the highest mean test R^2 .

```
lab/step7_conclusion.py · lines 17–22
```

```
17 chosen = table.features[search.best_row]
18 chosen_r2 = table.mean_r2[search.best_row]
19 chosen_rmse = table.rmse[search.best_row]
20 all_r2 = table.mean_r2.iloc[-1]
21 excluded = fit.numeric_features[len(chosen):]
22 weak_kept = [name for name in search.weak.added if name in chosen]
```

Step 7 · The three model families side by side

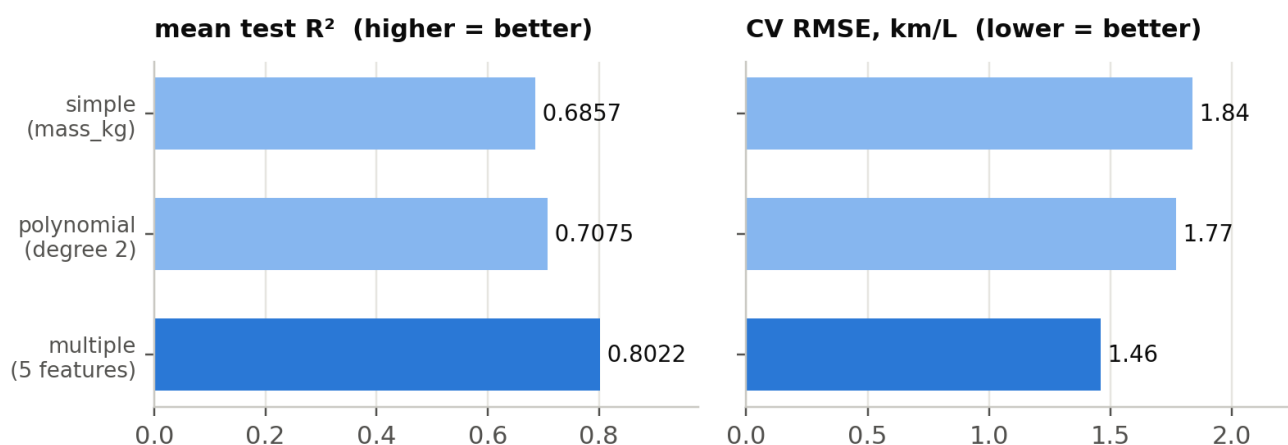


Figure 12 — The multiple regression wins on both measures. R^2 and RMSE are shown as two separate panels because they use different scales.

Model	Mean test R^2	CV RMSE (km/L)
Simple — <code>mass_kg</code>	0.6857	1.84
Polynomial — degree 2	0.7075	1.77
Multiple — 5 features	0.8022	1.46

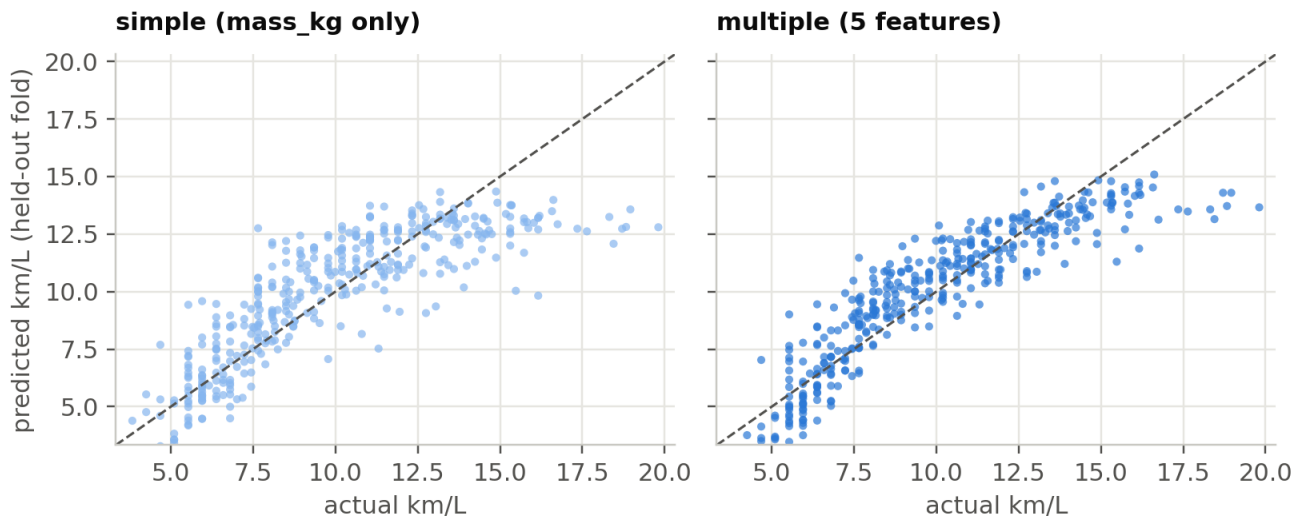
Step 7 · Held-out predictions vs reality (dashed = perfect prediction)

Figure 13 — Each car's prediction from the fold where it was held out. The multiple-regression points hug the diagonal much more tightly.

Why this model? Bending the `mass_kg` line into a curve gains only $+0.0218$ R^2 , while adding `year` gains $+0.1034$. Cars got more efficient from 1970 to 1982 even at the same weight, and no curve in weight alone can capture that. The typical error falls from 1.84 to 1.46 km/L.

Heads-up. The chosen model (R^2 0.8022) and the all-features model (R^2 0.8002) differ by only 0.002 — far less than the fold-to-fold spread (± 0.015). They are effectively tied, so preferring the one with fewer inputs is a judgement call on simplicity, not a clear win. The model was also trained only on 1970–1982 cars.

SECTION 9

How the written answers write themselves

The paragraphs in `results.md` are not typed by hand. Each is assembled from sentences whose wording depends on the numbers, so changing `RANDOM_STATE` and re-running updates the text as well as the tables.

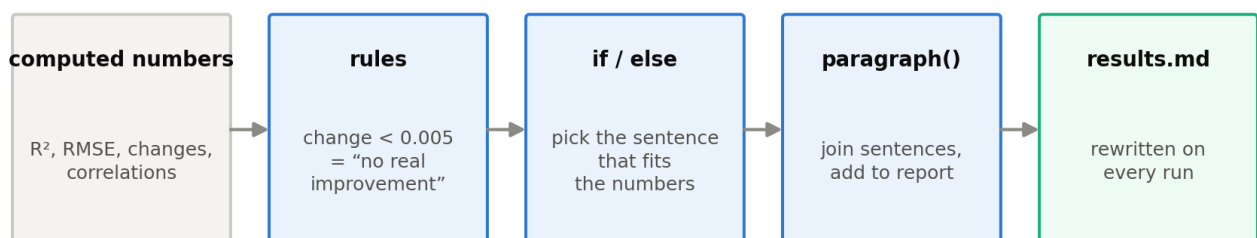


Figure 14 — From numbers to sentences.

lab/step5_multiple.py · lines 49–55 · finding the weak additions and the winner

```

49 additions = table.iloc[1:]
50 search = FeatureSearch(
51     table=table,
52     weak=additions[additions.change < MIN_GAIN],
53     top=additions.loc[additions.change.idxmax()],
54     best_row=table.mean_r2.idxmax(),
55 )

```

- **weak** keeps only the additions whose change is below **MIN_GAIN**.
- **top** is the single addition with the biggest gain; **best_row** is the model with the highest R^2 — the one Step 7 recommends.
- For every weak feature, the script looks up which earlier feature it overlaps with most (**feature_corr**) and names it in the answer.
- Safety nets: if a simple or polynomial model ever beats the Step 5 model, the script prints a **WARNING** and flags the conclusion to be rewritten; it also checks the conclusion is 150–250 words (this run: 222).

SECTION 10

Cheat sheet & sources

Metric formulas

Metric	Formula	In words
MAE	$(1/n) \sum y_i - \hat{y}_i $	average absolute error
MSE	$(1/n) \sum (y_i - \hat{y}_i)^2$	average squared error
RMSE	$\sqrt{\text{MSE}}$	typical error in the target's units
R^2	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	1 = perfect, 0 = no better than predicting the mean

Glossary

Term	Meaning
Feature / target	an input column / the value we want to predict
Fit	choose the θ 's that minimise squared error on the given rows
Train / test set	rows used for fitting / rows kept aside to check the model
K-fold CV	split into K parts; each part is the test set once; average the K scores
Overfitting	fitting noise in the training data, so test performance stops improving or drops
Multicollinearity	inputs that carry the same information (e.g. engine size and cylinders)
Pipeline	a chain of steps (scale → expand → fit) treated as one model
Module / package	one .py file / a folder of them (lab/) imported with from lab import ...
Frozen dataclass	a small read-only record of named values, used to pass results between steps

Sources & further reading

1. scikit-learn — LinearRegression
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
2. scikit-learn — Cross-validation: evaluating estimator performance
https://scikit-learn.org/stable/modules/cross_validation.html

3. **scikit-learn — Metrics and scoring (regression)**

https://scikit-learn.org/stable/modules/model_evaluation.html#regression-metrics

4. **scikit-learn — PolynomialFeatures**

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html>

5. **scikit-learn — Pipelines and composite estimators**

<https://scikit-learn.org/stable/modules/compose.html>

6. **scikit-learn — Underfitting vs. overfitting (example)**

https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html

7. **UCI Machine Learning Repository — Auto MPG (the classic 398-car dataset this lab's data resembles)**

<https://archive.ics.uci.edu/dataset/9/auto+mpg>

8. **pandas — DataFrame.corr**

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.corr.html>

Tools used to make this document: the charts were drawn with matplotlib from the output of the lab code itself; the PDF was typeset with ReportLab; code highlighting by Pygments.